MODULE 18

Mockito for Test Isolation

Learn how to use Mockito to create mocks, isolate dependencies, and test business logic with confidence.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 18 OVERVIEW

Learn how to use Mockito to create mocks, isolate dependencies, and test business logic in Java applications.

- Real code depends on databases, external services, and other classes. Mockito lets you replace those dependencies with controllable test doubles, so you can test business logic in complete isolation.

DURATION & LAB

1 Hour Theory

Mockito architecture, mocking, stubbing, verification.

2 Hours Lab

Mockito and Mocking.

Scenario

Test DefaultCustomerService in isolation by mocking CustomerRepository.

FOCUS
Test Isolation, Reliability, Maintainability

CORE SKILLS
Create mocks, stub methods, verify interactions

THE PAYOFF
Fast, deterministic, dependency-free unit tests

KEY TAKEAWAY Mockito lets you test business logic in isolation — no database, no external services, just your code under test.

WHY MOCKING MATTERS

Testing against real dependencies makes tests slow, flaky, and hard to control.

WITHOUT MOCKING

- Tests hit a real database or external service
- Slow, flaky, and order-dependent tests
- Hard to simulate error scenarios
- Failures may be caused by infrastructure, not code

VS

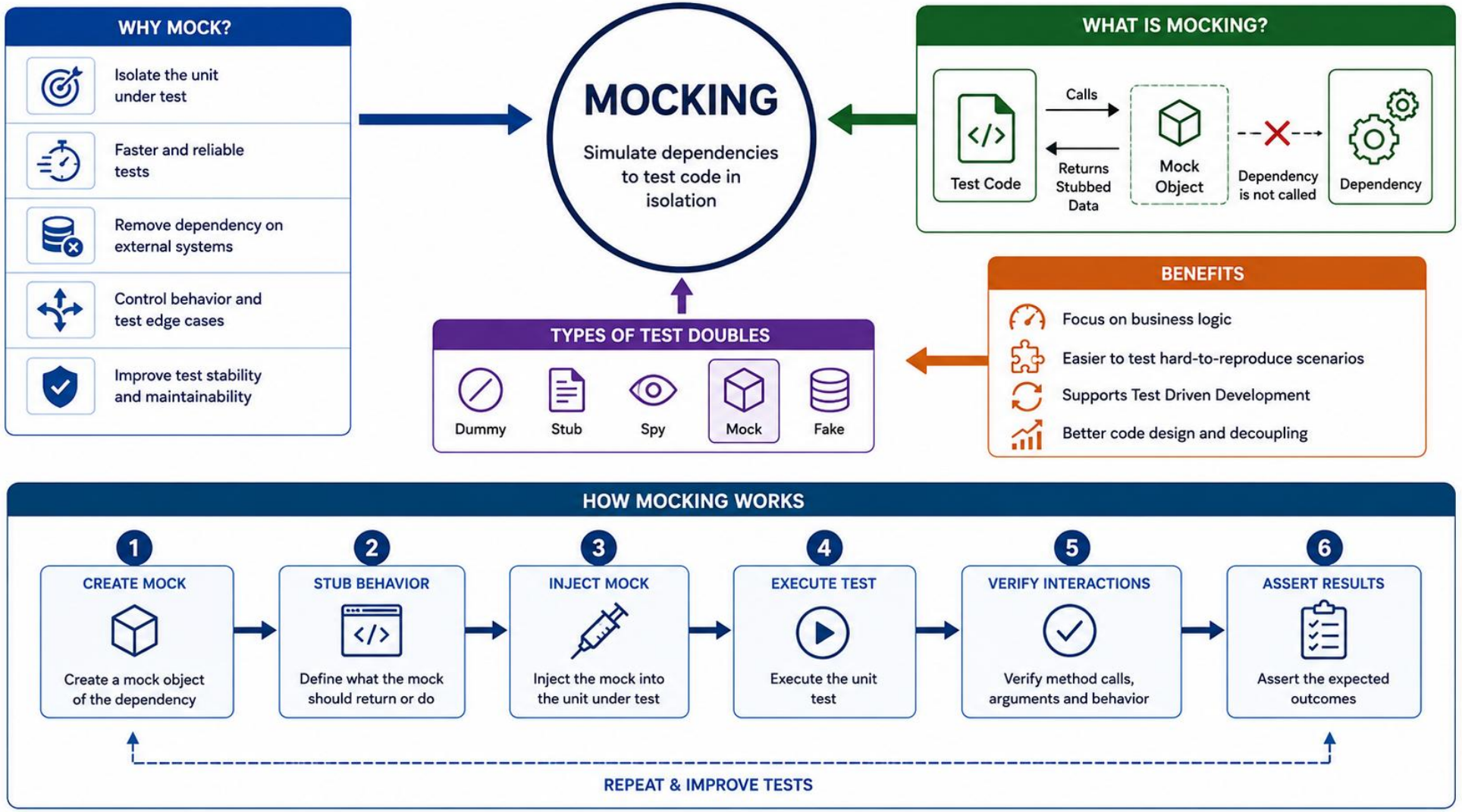
WITH MOCKING

- Dependencies replaced with controllable test doubles
- Fast, deterministic, isolated tests
- Easy to simulate success and failure scenarios
- Failures point directly to the unit under test

MOCKING ALLOWS US TO

- Speed up tests — no real calls to databases, APIs, or external systems.
- Improve reliability — tests don't fail due to external issues outside your control.
- Gain control over dependencies — define exactly what a dependency returns or does.
- Test edge cases easily — simulate exceptions, timeouts, and unusual scenarios.
- Encourage clean, loosely coupled code that's more maintainable and testable.

KEY TAKEAWAY Mocking isolates the unit under test so failures point to real bugs, not flaky infrastructure.



TEST DOUBLES OVERVIEW

"Test double" is the umbrella term for objects that stand in for real dependencies during testing.

TYPE	PURPOSE	EXAMPLE
`Dummy`	Passed but never actually used	A placeholder object required by a method signature
`Stub`	Returns predefined answers to calls	when(repo.findById(1L)).thenReturn(user)
`Mock`	Stub + records interactions for verification	verify(repo).save(any(User.class))
`Spy`	Wraps a real object, allows partial stubbing	spy(new OrderService())
`Fake`	A working, simplified implementation	An in-memory repository instead of a DB

Purpose: replace real collaborators with controlled objects so you can test one unit of code in isolation.
Note: These categories describe roles, not necessarily different libraries or object types. A Mockito mock may be used only as a stub in one test. A fake is usually a working implementation, not created by Mockito. A spy wraps or partially delegates to a real object. A mock is most useful when interaction verification matters.
Takeaway: Choose the simplest test double that gives the test the control it needs. Do not default to a mock when a real value object or small fake is clearer.

KEY TAKEAWAY Mockito creates configurable test doubles that can be used for stubbing, interaction verification, or partial delegation.

MOCKS VS. SPIES

A mock starts empty and returns defaults; a spy wraps a real object so only the methods you stub change.

EXAMPLE

```
List<String> mockList = mock(List.class);
when(mockList.size())
    .thenReturn(42);

List<String> real = new ArrayList<>();
List<String> spyList = spy(real);
spyList.add("mockito");

doReturn(100)
    .when(spyList).size();

verify(spyList).add("mockito");
```

KEY POINTS

- mock(List.class) creates an empty mock — every method returns a default value (0, null, empty) until stubbed.
- spy(real) wraps a real instance — unstubbed calls run the real method's actual logic.
- Use doReturn().when(spy) to stub a spy; when(spy.method()) can trigger real behavior first and may throw.
- Prefer a mock when no real logic is needed; use a spy only to override a few methods on real behavior.
- verify() works identically on both mocks and spies — interaction tracking doesn't depend on which you use.

KEY TAKEAWAY Reach for a mock first — use a spy only when you need most of an object's real behavior with one or two methods overridden.

TEST DOUBLES

Test doubles are stand-ins for real dependencies used in tests.

DUMMY

Passed around but never actually used.

Just to satisfy parameters.

STUB

Provides predefined answers to method calls.

Used to control outputs.

SPY

Wraps real object and records interactions.

Can verify calls and use real behavior.

MOCK

Expects specific interactions and can be programmed.

Verifies behavior and controls outputs.

FAKE

Working implementation with simplified logic.

Used instead of real dependency.

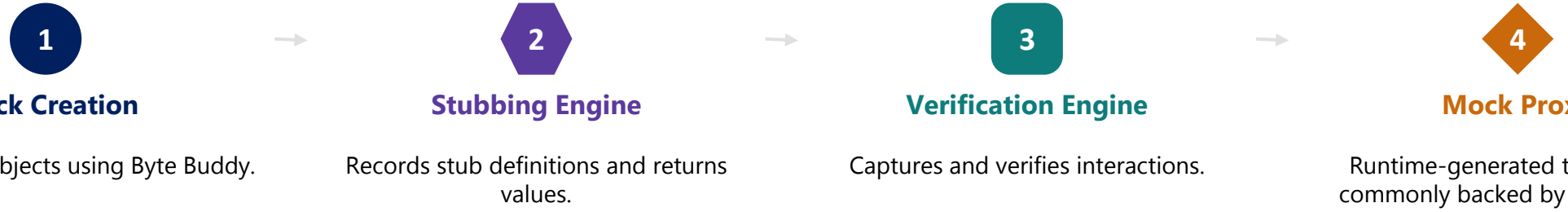
SCOPE OF USE

- Filling parameters not used
- Returning controlled data / values
- Partial mocking and interaction testing
- Behavior verification and expectations
- Replacing complex or external systems

TYPE	DUMMY	STUB	SPY	MOCK	FAKE
PURPOSE	Just to pass objects	Provide test data	Observe interactions	Verify interactions	Work like real but simpler
BEHAVIOR	No behavior	Returns programmed values	Calls real methods (can stub some)	Does not call real methods (unless stubbed)	Real implementation (simplified)
VERIFIES	✗ No	✗ No	✓ Yes	✓ Yes	✗ No
EXAMPLE USE	Method parameter not used	Repository returning user data	Service method calls and partial mocks	Verify repository.save() was called	In-memory database, fake email service

HOW MOCKITO WORKS

Mockito's core components work together in a repeatable cycle — from creating mocks to verifying interactions.



HOW MOCKITO WORKS — CORRECT STEP ORDER

1. Create mocks
2. Stub required behavior
3. Invoke the class under test
4. Assert outcome
5. Verify important interactions

Simple diagram: Test → Real Service → Mock Repository
Mockito generates runtime test doubles using its mock maker, commonly backed by Byte Buddy. The exact mechanism depends on the mocked type and configuration.

KEY TAKEAWAY Every Mockito test follows the same cycle: create mocks, stub required behavior, invoke the class under test, assert the outcome, then verify important interactions — stubbing always comes before invocation.

MOCKITO WORKFLOW IN CODE

Stubbing must happen before the method under test runs — reversing the order silently breaks the stub.

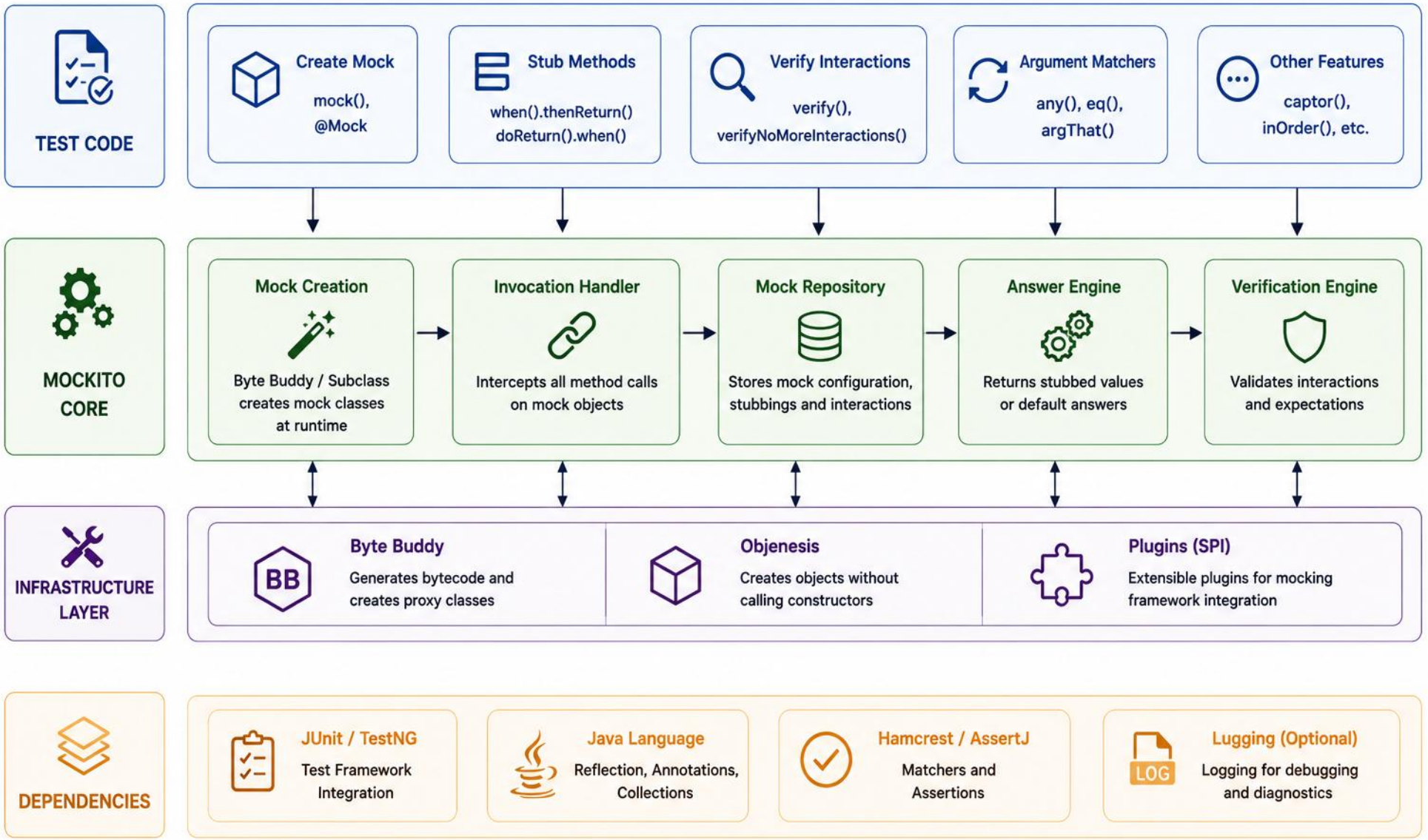
EXAMPLE

```
// WRONG: stub written after invocation
Customer result = service.getCustomer("CUS-1001");
when(repository.findById("CUS-1001"))
    .thenReturn(Optional.of(customer));
// result is null — the stub came too late

// RIGHT: create, stub, invoke, assert, verify
when(repository.findById("CUS-1001"))
    .thenReturn(Optional.of(customer));
Customer ok = service.getCustomer("CUS-1001");
verify(repository).findById("CUS-1001");
```

Note: The mock returns null for any unstubbed call. Calling getCustomer() before when() means the WRONG block above compiles and passes with a null result — silently.

KEY TAKEAWAY Define stubs before you call the method under test — Mockito can't retroactively apply a when() to a call that already happened.

MOCKITO ARCHITECTURE



CREATING MOCKS

Mockito provides several ways to create mock objects for your dependencies.

EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class DefaultCustomerServiceTest {
  @Mock
  CustomerRepository repository;
  CustomerValidator validator =
      new CustomerValidator();
  DefaultCustomerService service;
  @BeforeEach
  void setUp() {
    service = new DefaultCustomerService(
        repository, validator);
  }
}
```

KEY POINTS

- @Mock creates a mock object for a class or interface.
- Prefer explicit construction in @BeforeEach — pass mocks and real collaborators into the constructor directly for clarity.
- @InjectMocks is a convenience alternative: it creates the object under test and injects matching mocks automatically.
- @ExtendWith(MockitoExtension.class) is the simplest, primary way to enable annotations in JUnit 5.
- Alternative: Mockito.mock(Class) directly, or MockitoAnnotations.openMocks(this) — if used, close the returned AutoCloseable.
- Advanced: use MockSettings for advanced mock configuration (e.g., extra interfaces, custom answers).
- Convenience snippet: @InjectMocks DefaultCustomerService service; — declared once, wired automatically once all constructor dependencies are mocked or set.

KEY TAKEAWAY Mock dependencies whose real behavior would make the test slow, nondeterministic, difficult to control, or outside the unit's responsibility. Use real objects for simple, deterministic collaborators where that improves clarity.

@INJECTMOCKS & MOCK()

Two more ways to create mocks: automatic field injection, and a mock built without any annotation.

EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class DefaultCustomerServiceTest {
  @Mock
  CustomerRepository repository;
  @Mock
  CustomerNotifier notifier;

  @InjectMocks
  DefaultCustomerService service;
}

CustomerRepository repo =
    mock(CustomerRepository.class);
```

KEY POINTS

- @InjectMocks creates DefaultCustomerService and injects the @Mock fields above into its constructor automatically.
- Field injection only works when Mockito can match mocks to constructor parameters unambiguously.
- Mockito.mock(Class) creates a mock directly, with no annotation or extension — useful outside test classes.
- Prefer explicit constructor wiring (shown earlier) for clarity; use @InjectMocks for classes with many collaborators.
- Both approaches produce the same kind of mock — the difference is only how it's created and wired.

KEY TAKEAWAY @InjectMocks and Mockito.mock() are conveniences, not requirements — explicit constructor wiring stays the clearest choice for a few dependencies.

TRY IT YOURSELF — EXERCISE 1: WHEN TO KEEP REAL VALIDATOR

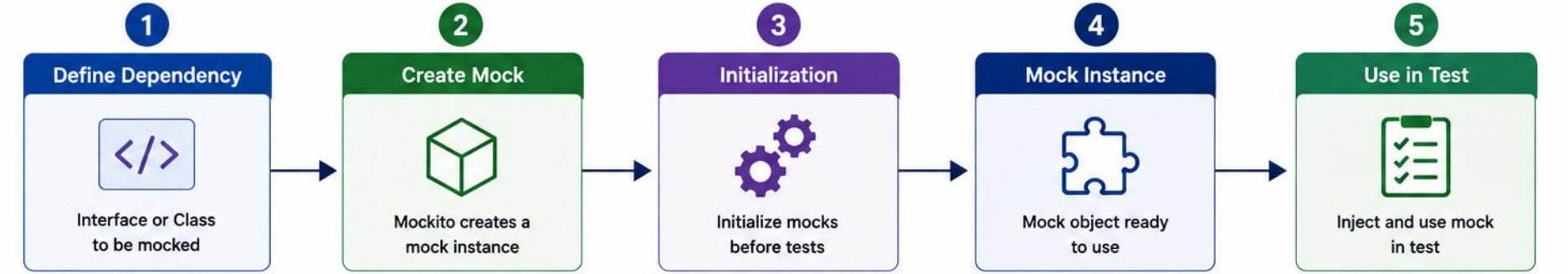
Reinforces: deciding which collaborators to mock vs. keep real — like the CustomerValidator you just saw wired in directly.

STEP	WHAT TO DO
Goal	Decide which activate() collaborator stays real and which gets mocked
File	mock-real-decisions.md (in examples/module-18-exercises/notes/)
Mock	CustomerRepository — an I/O boundary; slow, non-deterministic, outside the unit
Keep real	StatusValidator stays real when it is deterministic, fast, and pure domain logic
Deliverable	Repo-mock justified, validator-real justified, notifier-mock justified — a three-row decision table
Reference	exercises/exercise-01-keep-real-validator.md

```
$ cat mock-real-decisions.md
CustomerRepository  -> MOCK   // I/O boundary, slow / non-deterministic
StatusValidator     -> REAL   // pure, deterministic, fast
CustomerNotifier    -> MOCK   // avoid email/IO in unit tests
RULE: mock I/O and unstable deps; keep pure domain helpers real when cheap
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-keep-real-validator.md.

CREATING MOCKS




```
public interface
UserRepository {
  User findById(Long id);
  void save(User user);
}
```

```
@Mock
private UserRepository
 userRepository;

// or
UserRepository
 userRepository =
 mock(UserRepository.class);
```

```
@BeforeEach
void setUp() {
   MockitoAnnotations
     .openMocks(this);
}
```

userRepository (mock object)


```
@InjectMocks
private UserService
  userService;

// use in tests
userService.getUser(1L);
```

WAYS TO CREATE MOCKS

mock(Class<T>)

```
UserRepository repo =
mock(UserRepository.class);
```

@Mock Annotation

```
@Mock
UserRepository repo;
```

Mockito.mock()

```
UserRepository repo =
Mockito.mock(UserRepository.class);
```

@MockBean (Spring Boot Test)

```
@MockBean
UserRepository repo;
```

with Settings

```
UserRepository repo =
mock(UserRepository.class,
withSettings().name("repo"));
```

RESULT

STUBBING METHODS

Stubbing defines what a mock should return (or throw) when a method is called.

EXAMPLE

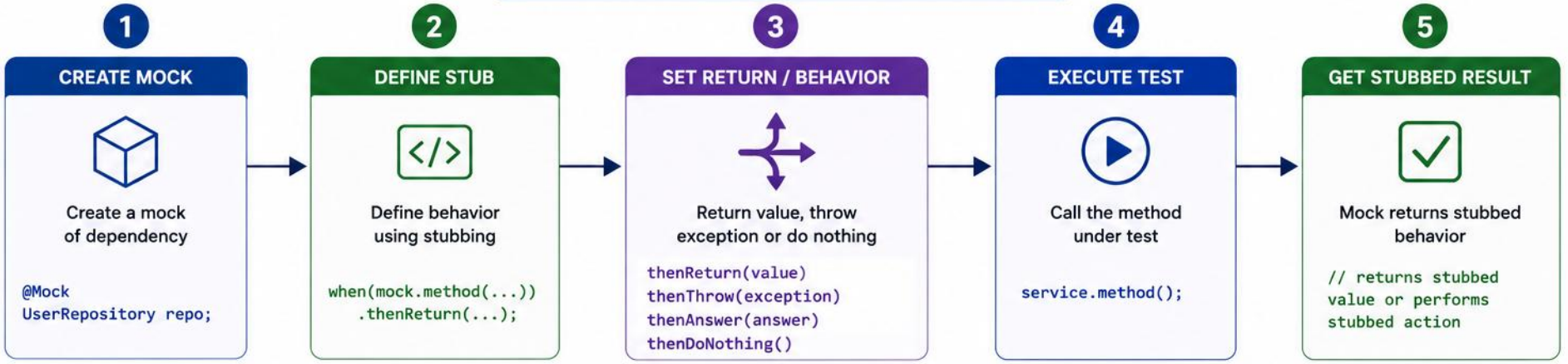
```
when(orderRepository.findById(1L))
    .thenReturn(Optional.of(order));
when(paymentGateway.charge(any()))
    .thenThrow(new PaymentException());

doReturn(value).when(spy).method();
doThrow(new NotificationException())
    .when(notificationService).send(any());

when(repository.findByStatusAndRegion(
    eq(ACTIVE), anyString()))
    .thenReturn(customers);
```

COMMON STUBBING METHODS

- when(mock.method()).thenReturn(value) — normal flow.
- when(mock.method()).thenThrow(exception) — exceptional flow.
- For void methods, use doNothing(), doThrow(), or doAnswer() — never doReturn().
- doReturn() is commonly useful for spies, or when when() would invoke real behavior.
- Use doNothing() only when it improves readability or is part of consecutive behavior. For ordinary mock void methods, no stubbing is required.
- Stub only what is necessary for the test.
- Advanced: thenReturn(v1, v2, v3) returns different values per call — can couple a test to invocation order.
- When one argument uses a matcher, all arguments in that invocation should normally use matchers.

KEY TAKEAWAY Stub only the behavior your test actually needs — over-stubbing makes tests brittle and hard to maintain.

STUBBING METHODS



STUBBING EXAMPLES

RETURN VALUE

```
when(repo.findById(1L))
 .thenReturn(Optional.of(user));
```

RETURN NULL

```
when(repo.findById(2L))
 .thenReturn(null);
```

THROW EXCEPTION

```
when(repo.save(any(User.class)))
 .thenThrow(new RuntimeException
  ("Save failed"));
```

DO NOTHING

```
doNothing()
 .when(emailService)
  .sendEmail(any());
```

CUSTOM ANSWER

```
when(repo.findById(anyLong()))
 .thenAnswer(invocation ->
  Optional.of(new User(
  invocation.getArgument(0))));
```

COMMON STUBBING PATTERNS

Specific Value

```
when(method(arg))
```
↓
```
thenReturn(value)
```

Any Value

```
when(method(any()))
```
↓
```
thenReturn(value)
```

Multiple Calls (Consecutive)

```
when(method())
```
↓
```
thenReturn(val1, val2, val3)
```

Conditional Answer

```
when(method())
```
↓
```
thenAnswer(invocation -> { ... })
```

Void Method

```
doThrow(exception)
```
↓
```
.when(mock).voidMethod();
```

STUBBING BENEFITS

- Control output and behavior
- Isolate unit under test
- Test edge cases easily
- Predictable and reliable tests
- Support TDD and clean design

VERIFYING INTERACTIONS

Verification confirms that the unit under test called its dependencies as expected.

METHOD	PURPOSE	EXAMPLE
`verify(mock).method(args)`	Confirms method was called once with args	verify(repo).save(user)
`verify(mock, times(n))`	Confirms method was called exactly n times	verify(repo, times(2)).save(any())
`verify(mock, never())`	Confirms method was never called	verify(gateway, never()).charge(any())
ADVANCED VERIFICATION		
`verify(mock, atLeastOnce())`	Confirms method was called at least once	verify(notificationService).sendCustomerActivated(customerId);
`inOrder(...).verify(...)`	Confirms methods were called in sequence	inOrder.verify(repo).findById(1L)
`verifyNoMoreInteractions(mock)`	Confirms no other interactions occurred	verifyNoMoreInteractions(userRepository)

```
ArgumentCaptor<Customer> captor =
    ArgumentCaptor.forClass(Customer.class);
verify(repository).save(captor.capture());
Customer saved = captor.getValue();
assertEquals(ACTIVE, saved.status());
```

Captor: Use a captor when the argument itself is an important observable outcome. Do not use it when a direct argument equality check is enough.

Results first: Assert the observable result first. Verify interactions only when they are part of the expected behavior. Often: result, exception, entity state, captured object, side effect.
atLeastOnce(): Prefer exact verification when the call count is part of the behavior. Use atLeastOnce() only when additional calls are intentionally acceptable.
verifyNoMoreInteractions(): Use it only when interaction completeness is part of the contract, such as security-sensitive side effects or ensuring no persistence occurs after rejection.

KEY TAKEAWAY Verify behavior, not implementation — confirm the important interactions happened, not every internal detail.

TRY IT YOURSELF — EXERCISE 2: STUB VS VERIFY

Reinforces: stubbing return values with when() vs. verifying side-effect calls with verify() — both covered above.

STEP	WHAT TO DO
Goal	Explain stubbing return values vs. verifying calls for activate()
File	lab18-stub-verify.md (in examples/module-18-exercises/notes/)
Stub	when(repo.findById("CUS-1002")).thenReturn(raviProspect) — arrange the input
Verify	verify(repo).save(activatedRavi) — assert the collaboration happened
Deliverable	A written stub example, a written verify example, and one contrast sentence
Reference	exercises/exercise-02-stub-vs-verify.md

```
$ cat lab18-stub-verify.md
when(repo.findById("CUS-1002"))   // STUB: arrange an input
    .thenReturn(raviProspect);
verify(repo).save(activatedRavi);  // VERIFY: assert a side effect
// Stubs feed inputs; verifies prove side-effect calls happened.
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-stub-vs-verify.md.

TRY IT YOURSELF — EXERCISE 3: ARGUMENTCAPTOR PREVIEW

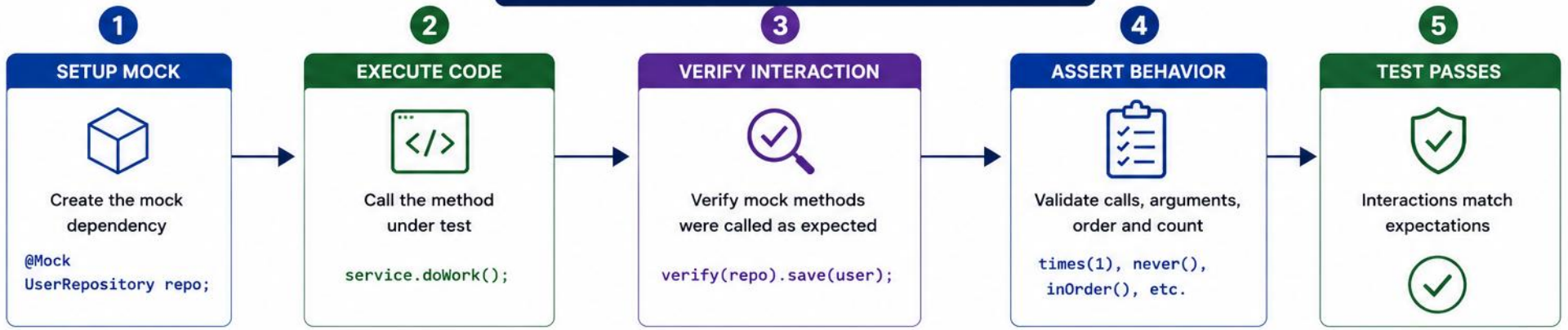
Reinforces: the ArgumentCaptor pattern shown above for inspecting a saved Customer's state, not just that save() ran.

STEP	WHAT TO DO
Goal	Sketch ArgumentCaptor steps for the saved Customer, on paper — no tests run yet
File	lab18-captor-sketch.md (in examples/module-18-exercises/notes/)
Declare	ArgumentCaptor<Customer> captor = ArgumentCaptor.forClass(Customer.class);
Verify + assert	verify(repo).save(captor.capture()); then assert captor.getValue().getStatus() is ACTIVE
Deliverable	A three-step captor sketch, ACTIVE asserted for Ravi, and a written pre-lab boundary note
Reference	exercises/exercise-03-argumentcaptor-preview.md

```
$ cat lab18-captor-sketch.md
ArgumentCaptor<Customer> captor = ArgumentCaptor.forClass(Customer.class);
verify(repository).save(captor.capture());
assertEquals(ACTIVE, captor.getValue().getStatus());
// Prepare for Lab 18; do not complete the full Mockito lab now.
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-argumentcaptor-preview.md.

VERIFYING INTERACTIONS



VERIFICATION METHODS

verify(mock)
Verify a method was called
```
verify(repo).save(user);
```

times(n)
Verify exact number of calls
```
verify(repo, times(2))
.findById(1L);
```

atLeast(n)
Verify minimum number of calls
```
verify(repo, atLeast(1))
.save(user);
```

atMost(n)
Verify maximum number of calls
```
verify(repo, atMost(3))
.save(user);
```

never()
Verify method was never called
```
verify(repo, never())
.delete(any());
```

verifyNoMoreInteractions()
Verify no other interactions occurred
```
verifyNoMoreInteractions
(repo);
```

ARGUMENT VERIFICATION

eq(value)
Match exact value
```
verify(repo).findById(eq(1L));
```

any()
Match any value
```
verify(repo).save(any(User.class));
```

argThat(predicate)
Match with condition
```
verify(repo).save(argThat(u ->
u.getName().startsWith("A")));
```

nullable(Class)
Match null values
```
verify(repo).findById(nullable(Long.class));
```

captures
Capture argument for assertions
```
ArgumentCaptor<User> captor =
ArgumentCaptor.forClass(User.class);
verify(repo).save(captor.capture());
```

COMMON VERIFICATION PATTERNS

Verify Single Call
```
verify(mock).method();
```

Verify Call Count
```
verify(mock, times(3)).method();
```

Verify No Call
```
verify(mock, never()).method();
```

Verify Order
```
InOrder inOrder = inOrder(mock);
inOrder.verify(mock).first();
inOrder.verify(mock).second();
```

Verify No More Interactions
```
verify(mock).method();
verifyNoMoreInteractions(mock);
```

MOCKING REPOSITORIES

Repositories abstract data access — mock them to control responses and isolate business logic from the database.

EXAMPLE

```
@Test
void getCustomer_whenCustomerExists_returnsCustomer() {
  when(repository.findById("CUS-1001"))
      .thenReturn(Optional.of(customer));
  Customer result =
      service.getCustomer("CUS-1001");
  assertAll(
      () -> assertEquals("CUS-1001", result.id()),
      () -> assertEquals("Amina Khan", result.name())
  );
  verify(repository).findById("CUS-1001");
}
```

Note: Mockito verifies service behavior, not persistence integration. Repository behavior still requires integration tests against the real persistence technology.

KEY TAKEAWAY Prefer real domain objects and value objects. Mock infrastructure boundaries and collaborators outside the behavior being tested.

MOCKING REPOSITORIES

1 Define Repository Interface

```
public interface
UserRepository {
  Optional<User> findById(Long id);
  List<User> findAll();
  User save(User user);
  void deleteById(Long id);
}
```

2 Create Mock

```
@Mock
private UserRepository
  userRepository;
```

3 Stub Repository Methods

```
when(userRepository.findById(1L))
  .thenReturn(Optional.of(user));

when(userRepository.save(any(User.class)))
  .thenReturn(savedUser);

when(userRepository.findAll())
  .thenReturn(Arrays.asList(user));
```

4 Use in Service Under Test

```
UserService service =
  new UserService(userRepository);

User result = service.getUser(1L);
service.createUser(newUser);
```

5 Verify Interactions

```
verify(userRepository).findById(1L);

verify(userRepository).save(any(User.class));

verify(userRepository, never())
      .deleteById(anyLong());
```

COMMON REPOSITORY STUBBING

Find By ID

```
when(repo.findById(1L))
  .thenReturn(Optional.of(user));

when(repo.findById(99L)
  .thenReturn(Optional.empty());
```

Find All

```
when(repo.findAll())
  .thenReturn(Arrays.asList(
    user1, user2
    ));
```

Save

```
when(repo.save(any(User.class)))
 .thenAnswer(invocation -> {
   User u = invocation.getArgument(0);
   u.setId(1L);
   return u;
 } ));
```

Delete

```
doNothing().when(repo)
  .deleteById(anyLong());

doThrow(new RuntimeException("DB Error"))
  .when(repo).deleteById(anyLong());
```

existsBy... / Count

```
when(repo.existsById(1L))
  .thenReturn(true);

when(repo.count())
  .thenReturn(5L);
```

BENEFITS OF MOCKING REPOSITORIES

- Isolates business logic from database
- Faster and reliable unit tests
- Control data and scenarios easily
- Test edge cases and error paths
- Verify interactions with persistence layer

MOCKING EXTERNAL SERVICES

Mock external service calls (payment gateways, email/notifications, third-party APIs) to test without real network calls.

Payment Gateways

- Stripe, PayPal, Braintree. Mock the gateway interface, not the SDK — test approved and declined responses.
- interface PaymentGateway {
- PaymentResult charge(...); }

Notification Services

- Email, SMS, push. Wrap the vendor client in an app-owned port; mock the port, not the SDK.
- interface CustomerNotificationGateway {
- void sendActivated(Customer c); }

Third-Party APIs

- Partner and internal REST APIs. Mock your own gateway interface, not deep framework chains (e.g., WebClient) — simulate timeouts and 4xx/5xx errors at that boundary.

BEST PRACTICE

- Return success and failure responses to test both the happy path and error-handling logic in your service.
- Blind spot: Use Mockito for application behavior. Use HTTP stubs, contract tests, or integration tests for protocol and payload correctness.

KEY TAKEAWAY Mock the application-owned boundary interface, not the vendor SDK or framework client — keep tests focused on your own logic, not third-party protocols.

STUBBING A PAYMENT GATEWAY

Mock the boundary interface and stub both the approved and declined payment paths.

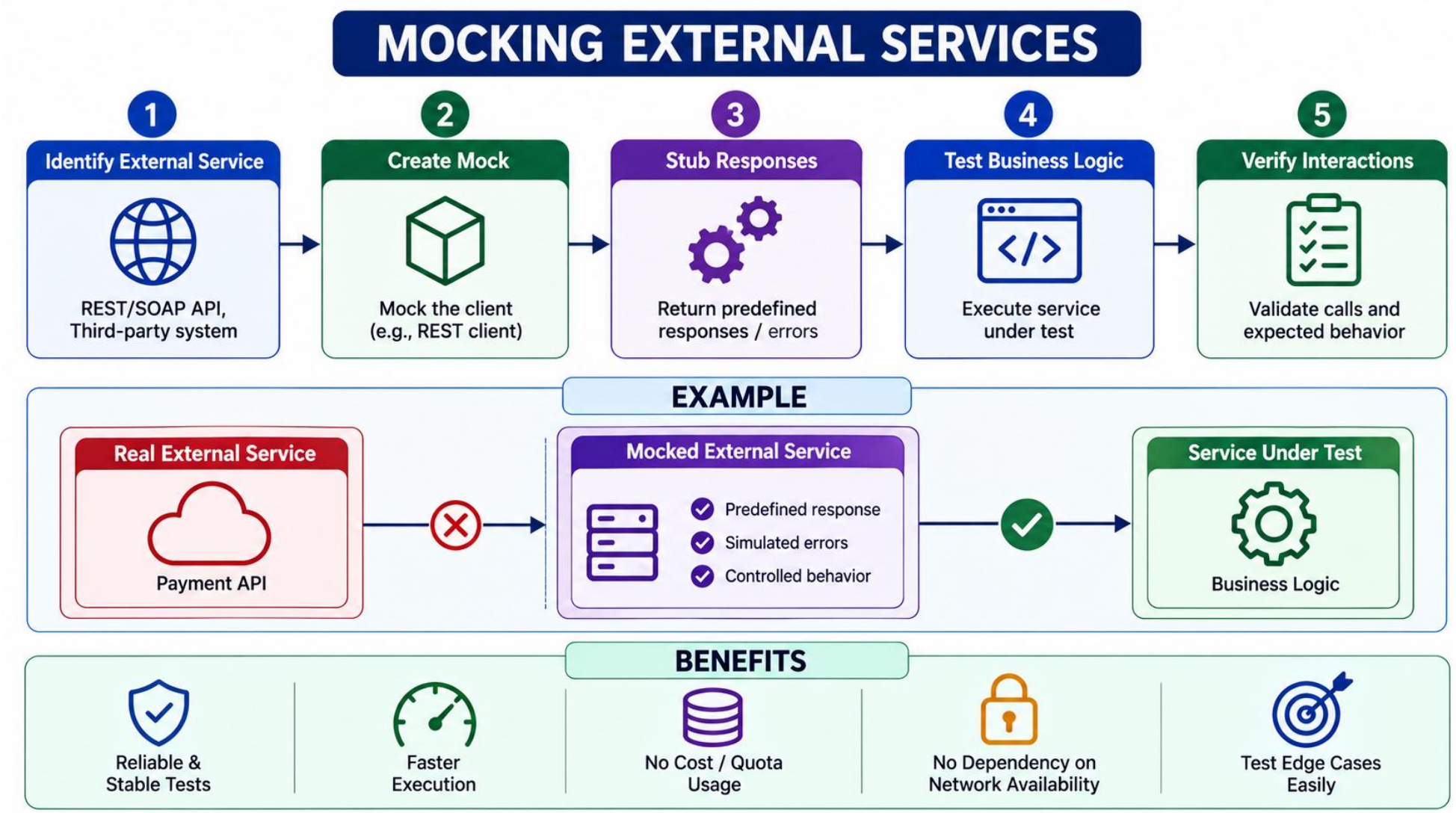
EXAMPLE

```
@Mock
PaymentGateway paymentGateway;

@Test
void charge_whenDeclined_throwsException() {
  when(paymentGateway.charge(any()))
      .thenThrow(new PaymentException(
          "Card declined"));

  assertThrows(PaymentException.class,
      () -> checkout.pay(order));
  verify(paymentGateway).charge(any());
}
```

KEY POINTS

- Mock the application-owned PaymentGateway interface — never the Stripe/PayPal SDK client directly.
- thenThrow() simulates a declined payment or network failure without making a real call.
- assertThrows() confirms the service translates the gateway exception into the expected behavior.
- any() matches any argument — use it when the exact charge request isn't the focus of this test.
- verify() confirms the gateway was actually invoked, not just that an exception was thrown some other way.

KEY TAKEAWAY Test both outcomes of an external boundary — thenReturn() for the happy path, thenThrow() for failures — without a single real network call.



MOCKING SERVICE LAYERS

Mock a service dependency when testing a controller or another service that calls it.

EXAMPLE

```
@Mock
private PricingService pricingService;

when(pricingService.calculateTotal(order))
    .thenReturn(new BigDecimal("199.99"));
```

WHY MOCK A SERVICE LAYER

- Test one layer at a time — the controller, not the service it calls.
- Avoid cascading real logic across multiple layers in a single test.
- Makes it easy to simulate edge cases the dependent service would produce.
- Mock an application service when testing a controller adapter. When testing one service that collaborates with another, consider whether the second collaborator is truly outside the unit or whether a broader component test would provide better confidence.

KEY TAKEAWAY Mocking service layers keeps each test focused on a single unit, even in a multi-layered architecture.

MOCKING SERVICE LAYERS

1 DEFINE SERVICE INTERFACE

Service interface with business operations

```
public interface UserService {
  User getUser(Long id);
  void createUser(User user);
  List<User> getAllUsers();
}
```

2 CREATE MOCK

Create mock of the service layer

```
@Mock
private UserService
  userService;
```

3 STUB SERVICE METHODS

Stub service layer methods

```
when(userService.getUser(1L))
  .thenReturn(Optional.of(user));

when(userService.getAllUsers())
  .thenReturn(Arrays.asList(user1,
   user2));
```

4 USE IN DEPENDENT LAYER TEST

Inject mock and test dependent components

```
@InjectMocks
private UserController
  userController;

userController.getUser(1L);
```

5 VERIFY INTERACTIONS

Verify service layer interactions

```
verify(userService).getUser(1L);

verify(userService, times(1))
  .createUser(any(User.class));

verifyNoMoreInteractions(userService);
```

COMMON STUBBING PATTERNS

RETURN VALUE

```
when(service.method())
  .thenReturn(value);
```

RETURN NULL

```
when(service.method())
  .thenReturn(null);
```

THROW EXCEPTION

```
when(service.method())
  .thenThrow(new
  RuntimeException("Error"));
```

RETURN COLLECTION

```
when(service.getAll())
  .thenReturn(Arrays.asList(
    item1, item2));
```

CONDITIONAL STUBBING

```
when(service.calculate(anyInt()))
  .thenAnswer(invocation -> {
    int arg = invocation.getArgument(0);
    return arg * 2;
  });
```

BENEFITS

- Isolates business logic
- Faster and reliable tests
- No dependency on complex services
- Control inputs and scenarios
- Easier testing of edge cases

ISOLATED SERVICE TEST WORKFLOW

A repeatable seven-step workflow for testing business logic against real domain data and mocked boundaries.

1. Arrange real domain data
2. Mock external boundaries
3. Stub only required behavior
4. Call the real service
5. Assert returned state or exception
6. Verify meaningful interactions
7. Confirm forbidden interactions did not occur

WHAT TO TEST

- Success paths (order placed, payment succeeds) and failure paths (payment declined, item not found) — both matter equally.

BEST PRACTICES

- Follow the AAA pattern: Arrange, Act, Assert.
- Test one business scenario per test.
- Use meaningful test data.
- Keep tests small, focused, and readable.

KEY TAKEAWAY Follow the same seven-step workflow for every isolated service test — from arranging real data to confirming forbidden interactions never happened.

ISOLATED WORKFLOW IN CODE

The same seven steps as one Mockito test — from arranging real data to confirming forbidden interactions.

EXAMPLE

```
@Test
void activate_whenProspect_activatesAndNotifies() {
  Customer ravi =
      Customer.prospect("CUS-1002", "Ravi Shah");

  when(repository.findById("CUS-1002"))
      .thenReturn(Optional.of(ravi));

  Customer activated = service.activate("CUS-1002");
  assertEquals(ACTIVE, activated.status());
  verify(repository).save(activated);
  verify(notifier).sendActivated(activated);

  verify(paymentGateway, never()).charge(any());
}
```

KEY TAKEAWAY Follow the same seven-step workflow for every isolated service test — arrange, mock, stub, call, assert, verify, then confirm forbidden interactions never happened.

TRY IT YOURSELF — EXERCISE 4: ACTIVATE TODOs (STARTER)

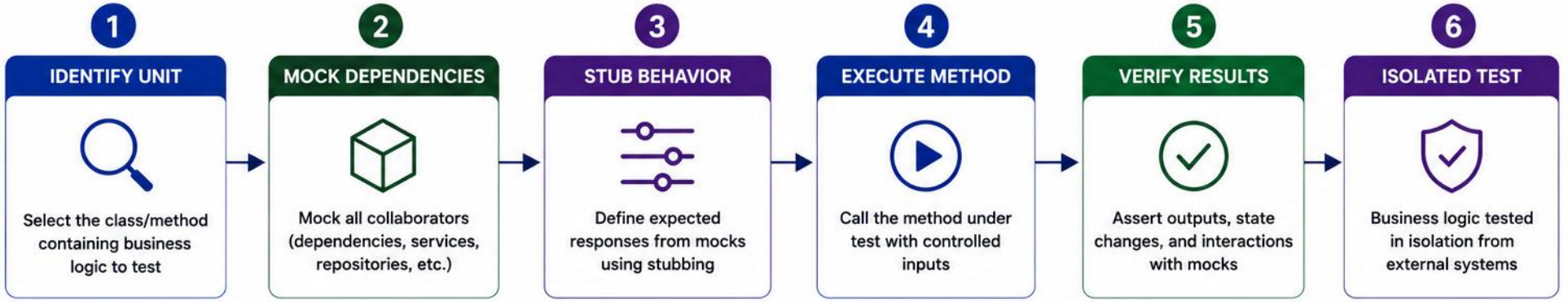
Reinforces: the seven-step isolated service test workflow you just saw, applied to the activate() interaction sequence.

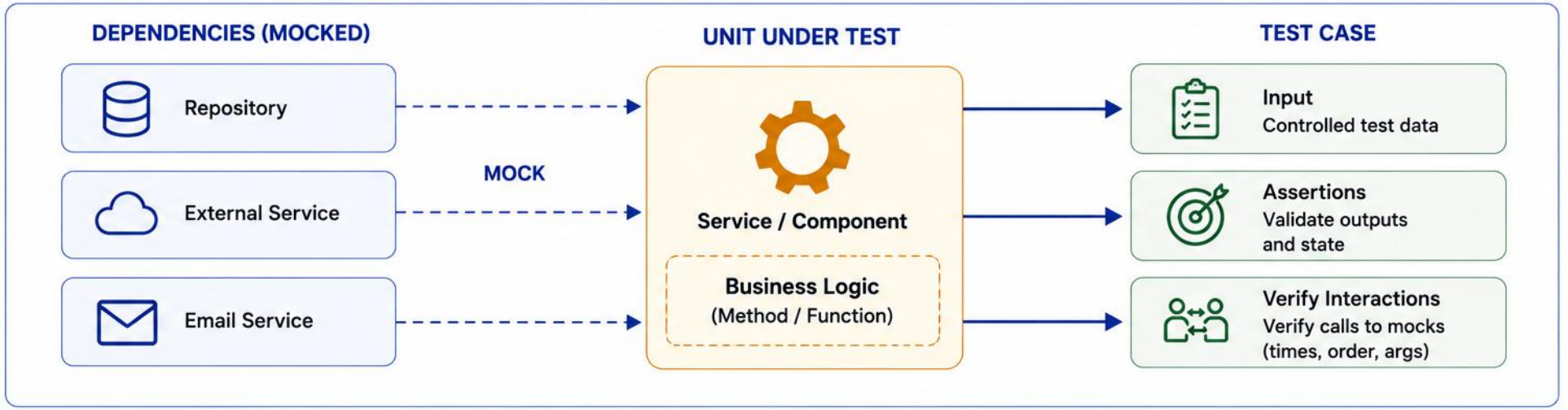
STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — complete six blanks in the activate() interaction sequence
File	lab18-interaction-todos.md (in examples/module-18-exercises/notes/)
Fill in	Six blanks: the customer id, the method name, two verify() calls, the final status, and the captor field
Self-check	Step 1's id must read CUS-1002; the final status in steps 5 and 6 must read ACTIVE
Deliverable	All six blanks replaced correctly, plus one sentence on the captor's benefit
Reference	exercises/exercise-04-fill-activate-interaction-todos.md

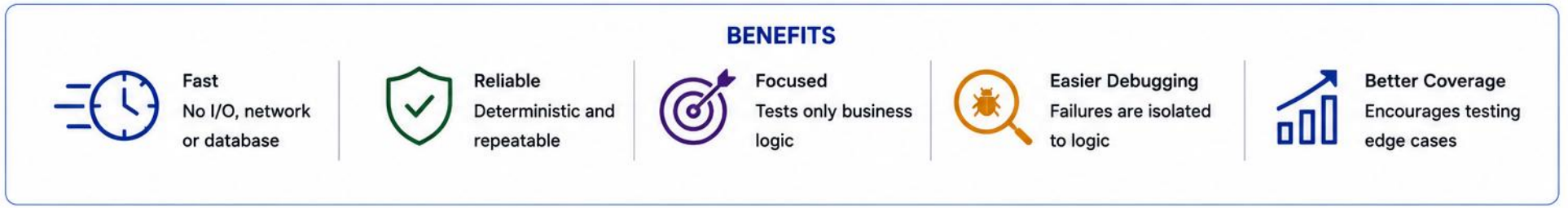
```
1) stub findById(_____) -> ravi PROSPECT
2) call service._____(...)
3) verify repo._____(customer)   4) verify notifier._____(...)  // if present
5) assert status _____
6) captor previews status field _____
```

TRY IT NOW Complete Exercise 4 before continuing (fill in all six blanks) — see exercises/exercise-04-fill-activate-interaction-todos.md.

TESTING BUSINESS LOGIC IN ISOLATION







COMMON MOCKING MISTAKES

Avoid these pitfalls to keep your mocked tests reliable and meaningful.

MISTAKE	PROBLEM	BETTER APPROACH
`Over-mocking`	Tests verify implementation, not behavior	Mock only true external dependencies
`Over-stubbing`	Stubs unused by the test add noise and confusion	Stub only what the test actually needs
`Verifying too much`	Brittle tests break on harmless refactors	Verify only interactions that matter
`Mocking value objects / DTOs`	Unnecessary and misleading	Use real instances for simple data objects
`Ignoring verifyNoMoreInteractions`	Missed unexpected calls go unnoticed	Use it when interaction completeness matters
`Mocking the class under test`	No real behavior is exercised	Instantiate the real class and mock only collaborators

KEY TAKEAWAY Mock dependencies, not the logic you're trying to test — mocking too much hides real bugs.

MISTAKE VS. FIX IN CODE

Verifying every call locks a test to today's implementation — verify only what the test's behavior depends on.

EXAMPLE

```
// MISTAKE: verifies every internal call
verify(repository).findById("CUS-1002");
verify(validator).validate(any());
verify(repository).save(any());
verify(notifier).sendActivated(any());
verify(auditLog).record(any());

// BETTER: verify only what the
// test's behavior depends on
verify(repository).save(activated);
verify(notifier).sendActivated(activated);
```

KEY POINTS

- The mistake block treats “verify everything” as a safety net — it actually locks the test to today's implementation.
- A harmless refactor (e.g., adding an audit log call) now breaks a test that never cared about auditing.
- The fix verifies only the two interactions the test's name and assertion actually depend on.
- This is the “Verifying too much” row from Common Mocking Mistakes: brittle tests break on harmless refactors.
- Rule of thumb: if deleting a verify() wouldn't let a wrong implementation pass, it isn't testing real behavior.

KEY TAKEAWAY Every verify() should protect a real behavior — if you can delete it without weakening the test, delete it.

TRY IT YOURSELF — EXERCISE 5: MOCKITO ANTI-PATTERNS

Reinforces: the common mocking mistakes table you just saw — now applied to Copilot-suggested Mockito code.

STEP	WHAT TO DO
Goal	List anti-patterns an AI assistant might suggest for CRM tests, and how to reject them
File	lab18-anti-patterns.md (in examples/module-18-exercises/notes/)
Table	Recreate the mistake/better table and add one row: mocking a String or enum status is silly
AI reject rule	Reject any suggestion that mocks CustomerService while the test is testing CustomerService
Deliverable	The extended table, the SUT-mock reject rule, and a written preference for real Amina/Ravi fixtures
Reference	exercises/exercise-05-anti-patterns.md

```
$ cat lab18-anti-patterns.md
Mock the SUT                      -> Mock collaborators only
Unnecessary stubbing               -> Stub only what the test uses
verifyNoMoreInteractions always    -> Use only when the interaction surface is critical
Mocking String/enum status         -> Silly — just use the real value
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-anti-patterns.md.

TRY IT YOURSELF — EXERCISE 6: LAB 18 PREP CHECKLIST

Reinforces: everything from Exercises 1–5 — confirm you're ready before Lab 18 begins.

STEP	WHAT TO DO
Goal	Confirm Mockito pre-lab prep is complete without starting Lab 18 itself
File	lab18-prep-checklist.md (in examples/module-18-exercises/notes/)
Confirm	Stub/verify note, mock/real decision table, anti-pattern sheet, and interaction TODOs all exist
Boundary	Write: "Pre-lab only — prepare for lab; do not complete full Lab 18."
Deliverable	A completed readiness checklist, captor preview confirmed, and the Lab 19 pointer noted
Reference	exercises/exercise-06-lab18-prep-checklist.md

```
[x] Exercise 2 - stub vs verify note saved
[x] Exercise 1 - mock/real decision table saved
[x] Exercise 5 - anti-pattern sheet saved
[x] Exercise 4 - interaction TODOs filled
[x] Exercise 3 - captor sketch saved
```

TRY IT NOW Complete Exercise 6 before Lab 18 — see exercises/exercise-06-lab18-prep-checklist.md.

LAB OVERVIEW – MOCKITO AND MOCKING WITH AI ASSISTANCE

Use Mockito to create mocks, stub methods, verify interactions, and test business logic in isolation.

SCENARIO

- Isolate DefaultCustomerService (Northstar CRM) by mocking CustomerRepository — keep CustomerValidator real so uniqueness rules still run.
- Verify find/save interactions, capture saved entities with ArgumentCaptor, and prove not-found paths never call save.

TECH STACK

ITEM	VALUE
`Language`	Java 21
`Testing`	JUnit 5, Mockito
`Build`	Maven
`IDE`	IntelliJ IDEA

KEY TAKEAWAY By the end of this lab, you'll write effective unit tests using Mockito and apply mocking techniques in real projects.

LAB TASKS AND DELIVERABLES

Complete these tasks to gain hands-on experience with Mockito — each builds on the previous one.

#	TASK	KEY ACTIVITIES	FOCUS AREA
1	Create Mocks	Use @Mock for each dependency; initialize with @ExtendWith(MockitoExtension.class).	Mock setup
2	Stub Methods	Use when().thenReturn()/thenThrow(); stub only what's necessary.	Behavior definition
3	Verify Interactions	Use verify(mock).method(), times(), never(), atLeastOnce().	Interaction verification
4	Mock Repositories	Stub scenario-by-scenario: existing lookup, missing lookup, save — only what the service actually calls.	Data access isolation
5	ArgumentCaptor & BDD	Capture the saved Customer; assert ID, name/email, status, and a single save. Add CustomerServiceBddMockTest using given()/then().should().	Captor + BDD style
6	Prove Isolation	verify(repository, never()).save(any(Customer.class)) and the thrown exception for CUS-9999 (or verifyNoInteractions(repository)). Run individually, as a suite, and reordered.	Negative paths & docs

SUCCESS CRITERIA

- CustomerServiceMockitoTest and CustomerServiceBddMockTest with a mocked CustomerRepository; verified interactions, ArgumentCaptor, and never().save() on not-found; clean, readable test code.

AI ASSISTANCE (OPTIONAL)

- Use GitHub Copilot to draft a mock setup, then review it: reject drafts that mock the class under test, leave unused stubs, or add Thread.sleep. Log: the prompt used, whether the suggestion was accepted or rejected and why, and any hallucinated API or unnecessary stub identified.

KEY TAKEAWAY Apply Mockito to create clean, isolated, reliable unit tests that validate behavior, not implementation.

MOCKITO TEST: DEFINITION OF DONE

Use this checklist to confirm an isolated Mockito test is complete before it ships.

1. Real class under test is instantiated
2. Unit boundary is clear
3. Simple domain objects remain real
4. Only external or controlled collaborators are replaced
5. Stubs are minimal and scenario-specific
6. Observable result is asserted
7. Important interactions are verified
8. Forbidden side effects are verified where relevant
9. Captors are used only when arguments matter
10. No unused stubs remain
11. Test does not depend on invocation order unless order is contractual
12. Repository and network integration are tested elsewhere
13. AI-generated Mockito code has been reviewed

KEY TAKEAWAY Testing in isolation leads to faster, more reliable, and maintainable tests that focus on behavior, not implementation.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Mockito and test isolation.

1. A test mocks Customer, CustomerValidator, CustomerRepository, and DefaultCustomerService itself. What is the primary problem?

A. Missing @ExtendWith(MockitoExtension.class)
B. The mocked class means no real logic runs.
C. Too many dependencies are mocked at once
D. CustomerValidator should not be mocked

2. A service returns the correct result, but the test verifies six internal repository calls not part of its public behavior. What risk does this create?

A. The test will fail intermittently
B. A brittle, implementation-coupled test.
C. The test runs slower than necessary
D. Mockito throws an UnnecessaryStubbingException

3. What does @InjectMocks do?

A. Creates a mock for the class under test
B. Injects created mocks into the real object under test
C. Verifies method calls
D. Starts a database connection

4. Best way to verify a method was never called?

A. verify(mock, never()).method()
B. verify(mock, times(0))
C. assertNull(mock)
D. when(mock).never()

KEY TAKEAWAY Mockito's @Mock, when()/thenReturn(), and verify() form the core toolkit for isolated unit testing.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. What is a common mocking mistake?

A. Stubbing only what's needed
B. Mocking every dependency including value objects
C. Verifying important interactions
D. Using meaningful test names

6. Why mock external services like payment gateways?

A. To slow down the test suite
B. To avoid real network calls and simulate failures easily
C. To make tests depend on live systems
D. To replace unit tests entirely

7. What should you verify in a well-designed test?

A. Every internal method call
B. Only interactions that matter to the test's purpose
C. Private field values
D. Nothing — verification is optional

8. What is the main benefit of testing in isolation?

A. Tests run against production data
B. Fast, deterministic tests that pinpoint real bugs
C. Tests require a live database
D. Tests take longer but are more thorough

ANSWER KEY

- **1-B 2-B 3-B 4-A 5-B 6-B 7-B 8-B**

KEY TAKEAWAY Mockito enables test isolation by mocking dependencies — the result is faster, more reliable, more maintainable tests.

"Test the unit, not the universe around it."

MODULE 18 COMPLETE

Mockito for Test Isolation

You can now use Mockito confidently to write clean, isolated unit tests that validate behavior, not implementation.